TÉCNICO LISBOA

Mobile Three-Dimensional Visualization and Animation

MEIC

Assignment 1 — OpenGL Racing Game on a Table

Authors:
Francisco Nascimento (106559)
Miguel Baptista (107312)
Gonçalo Guerreiro (120722)

Group 4

2026/2027 — 1st Semester, P1

1 Introduction

This assignment consisted of developing an interactive 3D racing game in modern OpenGL with GLSL shaders, built on the AVTM template. The player drives a car along a circuit built with road blocks on top of a table, while avoiding the *Cheerios* that delimit the track, the *butter* pieces and the *oranges* that roll across the table. This report summarises, for each feature, which techniques were used and how, with code excerpts and screenshots of the resulting scenes. Controls: `W/A/S/D` drive, `1/2/3` cameras, `N` directional light, `C` candles, `H` headlights, `F` fog, `M` rear-view, `O` OBJ car, `B` bounding boxes, `P` pause, `R` start/restart.

2 Graphic Modelling

All geometry is built from the template primitives (cube, sphere, cylinder, cone, torus), created once as VAOs and stored in `renderer.myMeshes`. Each object reuses one of these meshes with its own model matrix, using the matrix stack of the `gmu` class (`pushMatrix`/`popMatrix`, `translate`, `rotate`, `scale`).

The table is a scaled cube and the road is a closed circuit defined by eight path points, where each segment becomes a flat cube block. Cheerios are tori spaced along an outer and an inner ring around the road, butters are flattened cubes, oranges are spheres and the point orbs are translucent spheres.

The car is a hierarchical model: a single translation/rotation/scale places the whole car, and each part (body, cabin, seats, steering wheel, tyres, bumpers, lights, spoiler, windows, mirrors) has its own local transform inside it. Since the template cube spans $[0,1]^3$, each box is centred with `translate(-0.5,-0.5,-0.5)` after scaling.

OBJ support (Assimp). To import an `.obj` model, the code was adapted from the provided Assimp demo. In `buildScene()` the file is imported into an Assimp scene and converted into `MyMesh` objects, with their VAOs, materials and textures, by `createMyMeshFromAssimp()`. A new function, `renderMeshFromAssimp()` in `renderer.cpp`, sends the matrices and material to the shader, binds each mesh's textures to their texture units and draws it with `glDrawElements()`. `drawCarModel()` iterates over all the loaded meshes, applying the car's position, rotation and scale to each one, and is used both for the car and for its reflection. Key `O` switches between the OBJ car and the hand-modelled one, and the model's bounding box (`get_bounding_box`) updates the collision box.

```
// Import .obj after all the original meshes
std::string filepath = "assets/models/minicooper.obj";
// import 3D file into Assimp scene graph
if (!Import3DFromFile(filepath, importer, scene, scaleFactor))
    exit(0);
// creation of MyMesh array with VAO geometry and material and array
// of texture objects for the model input by the user
createMyMeshFromAssimp(scene, &renderer);
```

Listing 1: Importing the OBJ model and creating its meshes (`buildScene`).



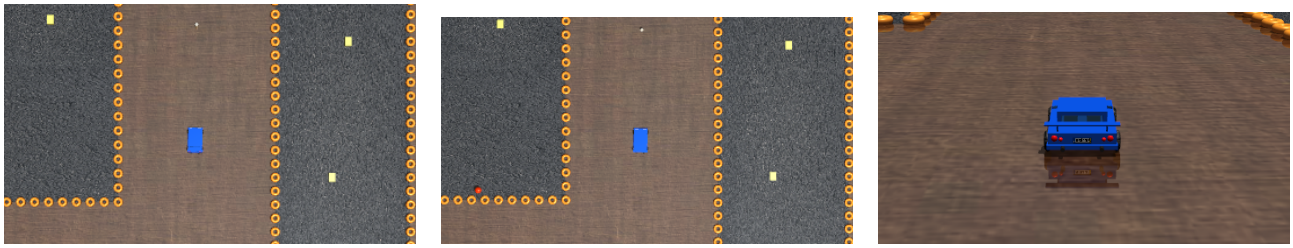
(a) Hierarchical car, front. (b) Hierarchical car, back. (c) Imported OBJ car.

Figure 1: Graphic modelling of the car.

3 Cameras

Three cameras were implemented: (1) a fixed top camera with orthographic projection, (2) a fixed top camera with perspective projection and (3) a perspective camera that follows the car. A set of global flags (`orthoProj`, `perspProj`, `fixedCamera`, `trackingCamera`) selects the active one. Pressing `1`, `2` or `3` updates the flags and calls `updateProjection()`, which rebuilds the projection matrix with `mu.ortho(...)` or `mu.perspective(...)` for the current window ratio. The same code is used in `changeSize()`, so the aspect ratio stays correct when the window is resized.

The top cameras are placed with `setCameraOver()` (spherical coordinates with $\beta = 90°$) and use $(0, 0, -1)$ as the up vector. The follow camera, `setCameraBehind()`, uses spherical coordinates centred on the car, with $\alpha = 180° + $ `carAngle` $+$ `cameraOrientation`, so it rotates with the car as in a racing game; dragging with the left mouse button adds an extra orientation offset and the right button/wheel changes the distance.



(a) Key 1: ortho. (b) Key 2: perspective. (c) Key 3: follow car.

Figure 2: The three cameras.

4 Movement of the Game Elements

Car. The car moves with acceleration instead of a fixed step. `W`/`S` apply an acceleration of ± 6 units/s^2; with no key pressed, a deceleration brings the velocity to exactly zero (without crossing it). Speed is clamped to 25 units/s, and `A`/`D` rotate the car at 75°/s only while it moves, with the steering inverted when reversing. The heading gives the direction vector $(\sin\theta, 0, \cos\theta)$, and the position is integrated as $p \mathrel{+}= d \cdot v \cdot \Delta t$. All updates use the real elapsed time between frames (clamped to 50 ms), so the speed does not depend on the frame rate.

Oranges. Each orange follows a straight line starting from a random border of the table, with one of three types (slow/green, normal/orange, fast/red) that differ in speed, radius and colour. Their speed grows with play time ($\times(1 + 0.05t)$, capped at $\times 5$), they roll with angular speed $\omega = v/r$ around the axis perpendicular to their direction, and they *hop* over Cheerios with a sine arc whose height and duration depend on the Cheerio size and the orange speed. When they leave the table they respawn; they bounce against butters and against each other by reflecting their direction around the contact normal ($d' = d - 2(d \cdot n)n$).

Other elements. Cheerios can be pushed by the car, and butters shake (oscillation around the Y axis, with decaying amplitude) for 0.8 s when hit. Point orbs disappear when collected and respawn after 20 s.

5 Lighting

All lighting is computed per fragment (Blinn–Phong) in `mesh.frag`; `mesh.vert` outputs the normal, eye vector and the fragment position in eye space (`fragPos`). All light positions are transformed to eye space on the CPU with the current view matrix before being sent as uniforms (`setGlobalLight`, `setCandlesLights`, `setSpotLights`).

- **Directional light** (key `N`): $l =$ normalize(`globalLightPos.xyz`), with $w = 0$, giving diffuse $\max(n \cdot l, 0)$ and specular $(n \cdot h)^{shininess}$.
- **Six point lights** (key `C`), representing candles placed on the table (small cylinders drawn at

their positions): `calculateCandleLight()` computes l from `lightPos` − `fragPos` and is called once per candle.

- **Two spotlights** (key `H`), the car headlights: `calculateSpotLight()` tests whether $\cos\theta = (-l \cdot s_d)$ is above the cutoff (0.93) and attenuates the intensity by $\cos^{60}\theta$. Their positions and direction are recomputed every frame from the car transform (`updateCarHeadlights`).

The diffuse and specular contributions of the three light types are summed and then combined with the material or texture colour in `main()`. The excerpt below shows the spotlight cone test and its attenuation.

```
float spotCos = dot(-l, sd);
if (spotCos <= spotCosCutOff) return vec3(0.0);    // outside the cone
float att = pow(spotCos, spotExp);                 // spotExp = 60
float intensity = max(dot(n, l), 0.0) * att;
```

Listing 2: Spotlight cone test and attenuation (`mesh.frag`).



(a) Directional. (b) Candles. (c) Headlights.

Figure 3: Directional, point and spot lights (each shown mostly on its own).

6 Collision Detection

Collisions use axis-aligned bounding boxes (AABB) with `checkOverlap()` on the three axes. Cheerios, butters, oranges and orbs have analytic boxes (e.g. $[x \pm r]$ for spheres). The car box is recomputed each frame by rotating the four corners of its local box in the XZ plane by the heading and taking the min/max, so it stays axis-aligned (`updateCarAABB`); with the OBJ car, the local box comes from the model's bounding box. Pressing `B` draws all boxes in wireframe.

The response depends on the object: with a *Cheerio* the car returns to its previous position and stops, and the Cheerio is nudged away; with *butter* the car also returns and is braked; hitting an *orange* or falling off the table costs a life and the car goes back to the start line; touching a *point orb* gives points. Penalties are applied only on the first frame of each contact (`wasHittingCheerio`/`wasHittingButter`), to avoid discounting every frame.

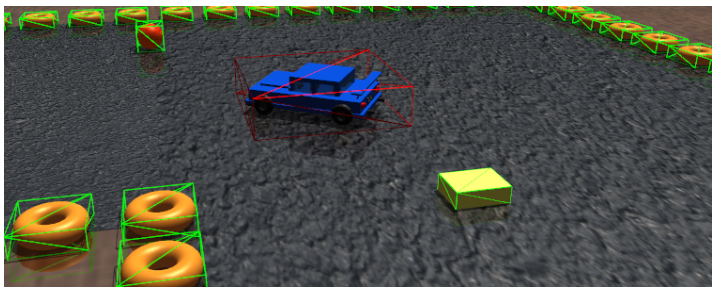


Figure 4: Bounding boxes of the car (red) and of the other objects (green).

7 Texture Mapping

Textures are loaded with DevIL into texture objects (`texture2D_Loader`) and bound to texture units when an object is drawn (`setTexUnit`), which also sets the corresponding `sampler2D` uniform. The

fragment shader chooses how to use them through the `texMode` uniform: **0** no texture; **1** diffuse colour modulated by the texel (road, oranges); **2** texel replaces the diffuse colour (finish line, tyres, licence plates); **3** multitexturing, the product of two textures (table: wood × table texture). Mode **4** is constant shading, used for the planar shadows. The texture coordinates come from the mesh VBO, and the results can be seen in the remaining figures of this report.

8 Transparency

The alpha of the output colour comes from the material diffuse alpha (`colorOut = vec4(finalColor, mat.diffuse.a)`). Translucent objects are drawn last, after all opaque ones, with `glEnable(GL_BLEND)`, `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)` and `glDepthMask(GL_FALSE)`, so that they test depth against the opaque scene but do not hide objects drawn after them. This is applied to the car glass (windshield, rear and side windows and mirrors) and to the point orbs (alpha 0.35, drawn last by `renderTranslucent`). The same mechanism makes the table and road semi-transparent over the planar reflection.

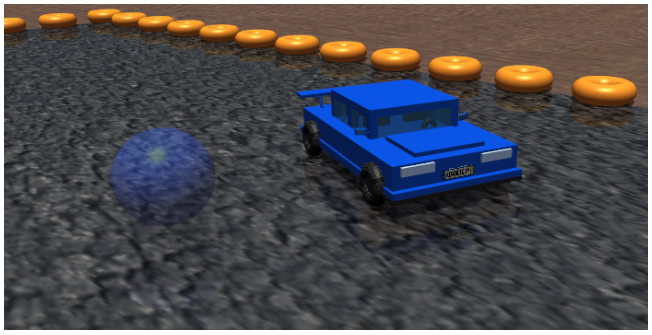
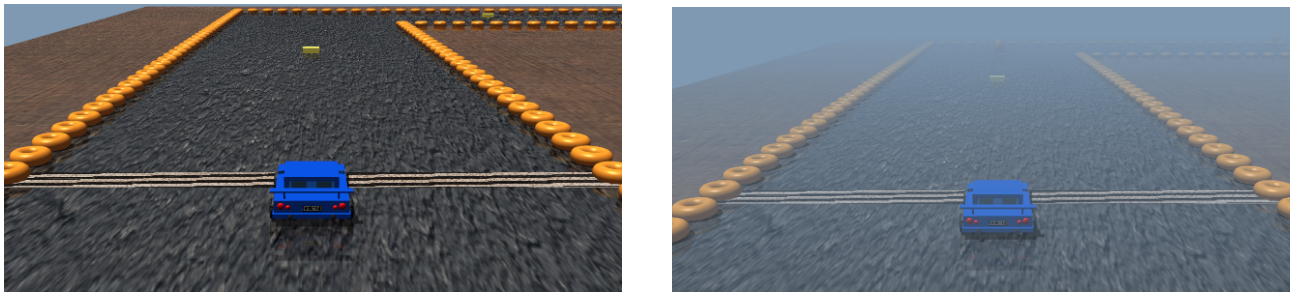


Figure 5: Transparent car windows and a translucent point orb.

9 Fog

Fog is computed in the fragment shader (key `F`) with an exponential law, $f = e^{-d \cdot 0.025}$, and the final colour is mix(`fogColor`, c, f). The shader supports both plane-based ($|z_{eye}|$) and range-based ($\|p_{eye}\|$) distance through the `depthFog` uniform; the plane-based mode is the one in use. The clear colour equals the fog colour, so distant objects blend smoothly with the background. For the planar shadows, the fog fades towards white, so shadows disappear in the distance instead of producing dark stains.



(a) Fog off.

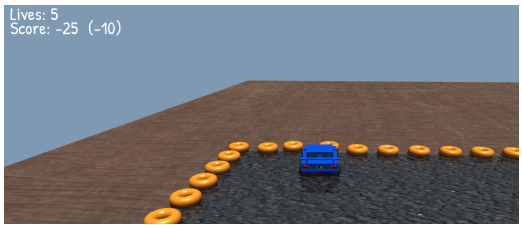
(b) Fog on.

Figure 6: Exponential fog.

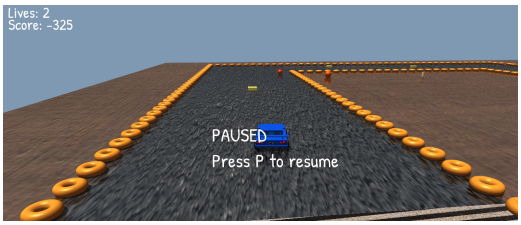
10 HUD, Pause and Game Rules

Text is rendered with `stb_truetype`: the font characters are packed into one texture (an atlas) and each character is drawn as a textured quad, with alpha blending hiding the background. The HUD is drawn last, with depth test disabled and an orthographic projection in window coordinates. It shows the lives (top-left), the score with the last variation (e.g. "−10") and central messages for the start menu, *Game Over* and *Paused*. `P` pauses (the update of the car and of the environment is skipped,

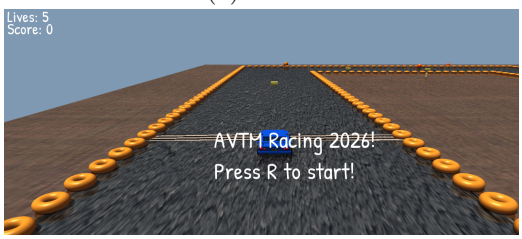
while rendering continues, so the camera can still be moved) and `R` starts or restarts the game. The player starts with 5 lives; Cheerio (−10), butter (−15) and lost life (−100) hits subtract points, and orbs add +100.



(a) HUD.



(b) Pause screen.



(c) Start Screen.



(d) Game Over screen.

Figure 7: HUD, start, pause and game over screens.

11 Rear-view Mirror

The rear-view mirror (key `M`, active with the follow camera) is a viewport in the top-right corner with 28% of the window size. It is drawn in these steps: (1) a dark rectangle, slightly larger, is drawn as a frame in screen coordinates; (2) the stencil buffer is filled with 1 over the mirror rectangle, using `GL_NEVER` with `GL_REPLACE` so that no colour is written; (3) colour and depth are cleared only inside the mirror using the scissor test; (4) with `glStencilFunc(GL_EQUAL,1,1)` and a viewport equal to the mirror area, the scene is rendered with a new perspective camera placed at the car and looking backwards. The lights are recomputed in eye space for this view before drawing, and the original state is restored at the end.

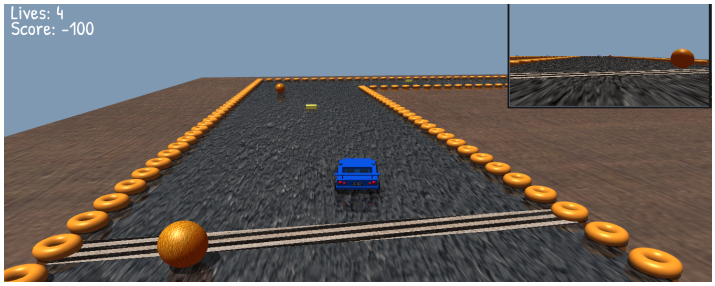


Figure 8: Rear-view mirror in the top-right corner.

12 Planar Reflections and Planar Shadows

Both effects use the stencil and blending mechanisms from the theoretical classes, on the table and on the road, and are only rendered when the camera is above the road (`camY > roadSurfaceY`).

Reflections. (1) The table and the road are drawn into the stencil buffer only (`GL_NEVER` with `GL_REPLACE`), with values 1 and 2 respectively. (2) With `GL_LEQUAL 1`, the objects above the road are drawn mirrored with $S(1,-1,1)$ about the plane $y = y_{road}$ and `glCullFace(GL_FRONT)`, as the mirror scaling flips the winding. The lights are mirrored too (position $y' = 2y_{road} - y$ and direction $y' = -y$) in `setSceneLights(true)`. (3) The table and road are redrawn semi-transparent (blending, depth writes off) only where the stencil equals 1 or 2, so the reflection is visible through them and never outside them. (4) The real objects are drawn at the end.

```
mu.pushMatrix(gmu::MODEL);
mu.translate(gmu::MODEL, 0.0f, planeY, 0.0f);     // mirror about the plane y = planeY
mu.scale(gmu::MODEL, 1.0f, -1.0f, 1.0f);
mu.translate(gmu::MODEL, 0.0f, -planeY, 0.0f);
glCullFace(GL_FRONT);                               // Flip face culling to reflection
drawObjectsAboveRoad(true);
glCullFace(GL_BACK);
mu.popMatrix(gmu::MODEL);
```

Listing 3: Mirrored rendering of the scene about the road plane.

Shadows. Blinn's technique projects every shadow caster on the plane with the directional light, using `shadow_matrix(plane, light)` multiplied into the model matrix. The stencil marks from the reflection step are reused: shadows are drawn only where the stencil is ≥ 1 (so never outside the table/road), and `GL_ZERO` on pass clears the mark, so each pixel is darkened at most once (no double darkening where two objects overlap). Depth test is disabled so the flat shadow does not fight with the floor, and blending with `(GL_DST_COLOR, GL_ZERO)` multiplies the existing colour. The geometry is drawn with constant shading (`texMode = 4`, flat grey), so it receives no lighting. Point orbs are excluded from the casters, since they are translucent.

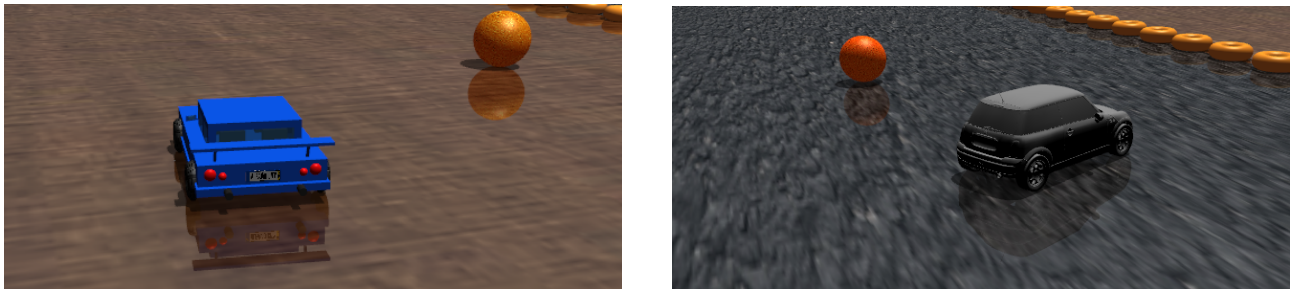


Figure 9: Planar reflections and shadows on the road and the table.

13 Extras

Beyond the baseline requirements, we added some elements to make the game more attractive: three orange types with different speed, size and colour, which hop over Cheerios and bounce on butters and on each other; collectable translucent point orbs that respawn after being collected; and a bounding-box debug view (key `B`), all described in the previous sections.

14 Conclusion

This assignment gave us a practical view of how a real-time 3D application is built on the modern OpenGL pipeline, from the CPU-side game logic to the shaders and fragment tests, and of how techniques such as blending and the stencil buffer can be combined to achieve effects like reflections and shadows.

15 Disclosure of AI Tools

During this assignment, we used Claude as an auxiliary tool, mainly to support code debugging and understanding of implementation details. However, all generated suggestions were manually reviewed, tested, and adapted to the project requirements before being included in the final code. Claude was also used during report preparation to help with LaTeX formatting and to improve wording in English. All generated content was validated by us.